

TRABAJO FINAL

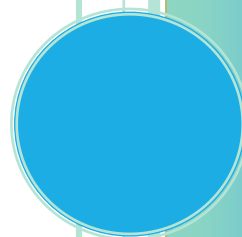
Sistemas Distribuidos y Paralelos

GRUPO 5

Ollier Gabriel | 02958/4

Joaquín Chanquía | 02887/7

04/06/2025



Trabajo Final

Sistemas Distribuidos y Paralelos

INDICE

Análisis de código secuencial -----	2
Solución en memoria compartida: Pthreads-----	3
Métricas-----	4
Verificación de resultados-----	5
Solución en memoria distribuida: MPI + Pthreads-----	6
Métricas-----	8
Verificación de resultados-----	9
Conclusiones-----	10

Análisis de código secuencial

La versión secuencial del problema de los N cuerpos se basa en un enfoque de simulación por pasos de tiempo discretos. En cada paso, se realizan dos operaciones fundamentales: el cálculo de las fuerzas gravitatorias entre los cuerpos y el movimiento de cada cuerpo según la resultante de dichas fuerzas. Ambas operaciones se implementan de forma directa siguiendo las leyes de Newton, utilizando estructuras de datos que representan la masa, posición y velocidad de cada cuerpo en el sistema.

Para el cálculo de fuerzas el programa recorre todos los pares posibles de cuerpos (i, j) con $i < j$ y aplica la ley de gravitación universal de Newton para calcular la magnitud y dirección de la fuerza mutua entre ellos. Esta fuerza se descompone en sus componentes cartesianas (x, y, z) , y se aplica tanto al cuerpo i como al cuerpo j con sentidos opuestos, aprovechando el principio de acción y reacción. Al calcular ambos efectos simultáneamente, se evita la duplicación de cálculos y se reduce a la mitad el número total de iteraciones necesarias.

Desde el punto de vista computacional, esta función tiene una complejidad de orden $O(N^2)$, ya que se debe considerar la interacción entre todos los pares de cuerpos en el sistema. Además, cada interacción involucra operaciones de raíz cuadrada, división y normalización vectorial, lo cual representa una carga computacional considerable. Esta etapa es la más costosa en términos de tiempo de ejecución, y por lo tanto es el principal objetivo de toda estrategia de optimización o paralelización.

En cambio, para el movimiento de cuerpos, una vez obtenidas todas las fuerzas acumuladas sobre cada cuerpo, el programa recorre secuencialmente el conjunto completo de cuerpos, actualizando sus velocidades y posiciones en función de las fuerzas acumuladas previamente. Cada cuerpo está almacenado en un arreglo contiguo en memoria, lo que permite acceder a sus datos de forma eficiente. Para cada cuerpo, se consultan las componentes de la fuerza total (X, Y) almacenadas en vectores auxiliares alineados por índice, y se calculan los nuevos valores de velocidad y posición mediante una integración temporal directa.

Los nuevos valores se escriben directamente en la misma estructura del cuerpo, sin crear copias ni estructuras intermedias. Esto simplifica la gestión de memoria y evita sobrecargas adicionales. Dado que los cuerpos se actualizan de manera completamente independiente entre sí, no hay dependencia de datos en esta etapa, y su complejidad computacional es lineal.

Solución en memoria compartida: Pthreads

A partir del diseño secuencial del algoritmo del problema de los N cuerpos, se desarrolló una solución paralela utilizando la biblioteca POSIX threads centrada a su ejecución en arquitecturas de memoria compartida con múltiples núcleos, con el objetivo de reducir los tiempos de ejecución aprovechando el hardware disponible.

La estrategia de paralelización implementada sigue el esquema SPMD (Single Program Multiple Data), donde todos los hilos ejecutan el mismo código pero sobre distintas porciones de los datos. Se utiliza una barrera de sincronización para asegurar que todos los hilos finalicen el cálculo de fuerzas para así poder hacer el movimiento de cuerpos, dado que esta segunda etapa depende de los resultados de la primera. Y luego de la función de movimiento de cuerpos se implementa otra barrera para que todos los hilos comiencen el siguiente paso en simultáneo con todos los cuerpos ya actualizados.

Función calcular fuerzas

La paralelización del cálculo de fuerzas se realiza asignando a cada hilo una porción de los cuerpos sobre los cuales calcular interacciones, siguiendo una estrategia por stripes. En esta técnica, cada hilo procesa cuerpos de forma intercalada, es decir, el hilo con identificador `tid` calcula las interacciones para los cuerpos con un $i = tid + kT$ ($k=0,1,2,\dots$), donde T es el número total de hilos. Esto permite distribuir el trabajo de forma equitativa y sin solapamientos, evitando conflictos de escritura. Las fuerzas resultantes se almacenan en una matriz de dimensión $T \times N$, donde cada fila corresponde a un hilo y cada columna a un cuerpo. De este modo, cada hilo escribe exclusivamente en su propia fila, lo cual garantiza la ausencia de condiciones de carrera. Las fuerzas que cada cuerpo recibe se acumulan en las posiciones `fuerza_total[tid][i]`, permitiendo que la suma final se realice de forma segura después de la sincronización. Esta estrategia mejora la localidad espacial, ya que cada hilo accede repetidamente a una región concreta de memoria (su propia fila), reduciendo los fallos de caché. Si bien la asignación por stripes puede generar cierto desbalance de carga, este efecto es mínimo cuando el número de cuerpos es grande, y el beneficio en términos de rendimiento y uso eficiente de la caché compensa ampliamente dicha irregularidad.

Función mover cuerpos

Una vez calculadas y sincronizadas las fuerzas, cada hilo se encarga de actualizar una porción exclusiva del conjunto de cuerpos. Para ello, se divide el arreglo de cuerpos en bloques disjuntos y cada hilo recibe un rango de índices que le corresponde procesar de forma exclusiva. Lo cual garantiza que no haya dos hilos modificando simultáneamente la misma posición de memoria. Este diseño elimina por completo las condiciones de carrera durante la etapa de

movimiento, ya que cada hilo trabaja sobre un subconjunto privado y no solapado del arreglo.

Para cada cuerpo de su rango, el hilo correspondiente acumula las fuerzas parciales que se encuentran distribuidas en las distintas filas de la matriz de fuerzas (las calculadas por todos los hilos) y las centraliza en una fila común. A continuación, con la fuerza total sobre cada cuerpo, se calculan las aceleraciones y se actualizan las velocidades y posiciones escribiendo en la estructura del cuerpo correspondiente. Esta etapa, además de ser naturalmente paralelizable, presenta un patrón de acceso secuencial a memoria, lo cual mejora la localidad espacial y reduce los fallos de caché. Al no requerir sincronización adicional ni mecanismos de exclusión mutua, el diseño logra un buen rendimiento y mantiene la consistencia en entornos de ejecución concurrente.

Métricas:

Para cada tamaño de unidades de procesamiento y para cada tamaño de problema se toman los tiempos de ejecución promedio de tres iteraciones para realizar un cálculo más realista.

Tiempos	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	11,32	45,44	181,35	728,67
4	3,23	12,81	50,88	204,48
8	1,74	6,74	25,98	101,75

Speedup	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1,00	1,00	1,00	1,00
4	3,50	3,55	3,56	3,56
8	6,51	6,74	6,98	7,16

Eficiencia	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1,00	1,00	1,00	1,00
4	0,88	0,89	0,89	0,89
8	0,81	0,84	0,87	0,90

El análisis de las métricas obtenidas para la versión paralela con Pthreads muestra un comportamiento muy favorable en términos de speedup y eficiencia. Los valores crecientes de speedup al aumentar tanto la cantidad de hilos como el tamaño del problema reflejan la aplicación de la ley de Gustafson, que considera que el tamaño del problema puede crecer con la cantidad de recursos disponibles. Esto se observa en la mejora de la eficiencia a medida que se incrementa N, lo cual indica que la fracción paralelizable del programa

domina y el overhead de coordinación se diluye en cargas mayores. Por otro lado, también puede analizarse el comportamiento bajo el enfoque de la ley de Amdahl, observando cómo varía la eficiencia al mantener el tamaño del problema fijo y aumentar la cantidad de hilos (por ejemplo, para $N = 512$ y $T = 4, 8$). Si bien se advierte una leve caída de eficiencia en ese caso, los valores siguen siendo altos (por encima del 80%), lo que sugiere que la fracción secuencial del código es baja. No obstante, al tener sólo dos valores discretos de T , el análisis según Amdahl es limitado y no permite observar claramente un punto de saturación. En términos de clasificación, podemos afirmar que la solución es débilmente escalable, ya que la eficiencia se mantiene o mejora al aumentar tanto T como N en conjunto. Además, en algunos escenarios, especialmente con problemas como $N = 4096$, la eficiencia se conserva incluso al incrementar los hilos, lo cual permite afirmar que el algoritmo se comporta como cercano a fuertemente escalable. En conjunto, las métricas obtenidas validan la efectividad de la paralelización implementada sobre un entorno de memoria compartida y multicore.

Verificación de resultados

Dado que el enunciado pedía una verificación de resultados decidimos corroborar estos generando una función que calcule la distancia promedio de cada punto en su posición final calculado en la simulación paralela con respecto a su contraparte calculado en la simulación en secuencial.

Distancia promedio al secuencial	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	0,000	0,000	0,000	0,000
4	0,038	0,666	0,069	0,567
8	0,093	0,750	0,055	0,454

Como puede observarse la distancia promedio obtenida entre todos los cuerpos para este código es muy pequeña lo que nos da indicio de un buen funcionamiento del código, no alcanzando para ninguna simulación una distancia promedio de más de una unidad de distancia.

Solución en memoria distribuida: MPI + Pthreads

Para abordar el problema de los N cuerpos en un entorno distribuido con múltiples núcleos por nodo, se implementó un algoritmo híbrido que combina dos niveles de paralelismo: MPI para dividir el trabajo entre procesos distribuidos en distintas máquinas, y Pthreads para dividir el trabajo con hilos dentro de cada máquina. Esta estrategia híbrida permite aprovechar de forma eficiente los recursos del clúster multicore del III-LIDI.

El diseño parte del código secuencial, y se enfoca en paralelizar de forma explícita sus dos funciones principales: el cálculo de fuerzas gravitatorias entre cuerpos y el movimiento de los cuerpos. Ambas operaciones, costosas en términos de cómputo, fueron adaptadas al modelo híbrido para ejecutarse en paralelo.

El conjunto completo de cuerpos se divide equitativamente entre los procesos MPI. Cada proceso trabaja con un bloque de $N_{loc} = N / P$ cuerpos, donde P es la cantidad total de procesos. A su vez, cada proceso lanza $T_{loc} = T / P$ hilos POSIX para procesar su bloque de cuerpos en paralelo. Los cuerpos se reparten de manera fija entre los procesos y luego se calcula la porción asignada a cada hilo, siguiendo una estrategia estática que evita comunicación innecesaria durante las fases internas de cálculo y movimiento. Esta doble división permite escalar la aplicación tanto agregando máquinas como agregando núcleos por máquina, cumpliendo con los tres escenarios propuestos en la consigna.

Algoritmo sistólico

Se utilizó un algoritmo sistólico para la implementación del código híbrido ya que este permitía que todos los procesos realicen cálculo y se comuniquen, lo cual aumenta la distribución de carga entre los procesos. Además, al combinar esta estrategia con pthreads se permite que cada hilo realice los cálculos necesarios sobre sus propios conjuntos de cuerpos y acelerar los cálculos internos que son requeridos para el funcionamiento del algoritmo.

Primera etapa del algoritmo sistólico

En la primera fase del paso de simulación, cada proceso MPI calcula las fuerzas internas entre los cuerpos que le han sido asignados. Esta operación es completamente local, por lo que se ejecuta en paralelo entre los hilos del proceso usando una estrategia por stripes, que distribuye equitativamente el trabajo sin solapamientos. Antes de la realización de este cálculo, el hilo principal ($tid = 0$) del proceso envía su bloque de cuerpos a todos los procesos de menor rank. Para finalizar esta sección se implementa una barrera que permite que todos los hilos continúen a la siguiente sección en simultáneo.

Segunda etapa del algoritmo sistólico

En la segunda fase, cada proceso recibe los bloques de cuerpos que habían sido enviados por los procesos con un rank mayor. Nuevamente el encargado de recibir las comunicaciones es el hilo principal y el resto de los hilos quedan esperando en una barrera hasta la finalización de esta comunicación. Luego de tener asegurados los datos se procede a realizar otra función de cálculo de fuerzas pero que esta vez realiza todas las combinaciones disponibles. Nuevamente cada hilo almacena las fuerzas que calcula en una estructura auxiliar tanto para las fuerzas de sus cuerpos como para las de ellos que recibe. Al terminar este cálculo de fuerzas se reducen los resultados (de forma paralela también) para luego ser enviados por el hilo con tid 0 al proceso que originalmente envió sus cuerpos.

Tercera etapa del algoritmo sistólico

Ya para la última etapa del algoritmo se realiza la recepción de las fuerzas calculadas por los procesos con un rank menor por parte del hilo con tid 0. Al tener las fuerzas se realiza la reducción de estas con las fuerzas internas ya calculadas en la etapa 1. Se aprovecha el uso de todos los hilos para la realización de esta reducción y se espera al finalizar nuevamente con una barrera.

Ya una vez acumuladas todas las fuerzas que actúan sobre un cuerpo se realiza el cálculo de su nueva posición y velocidad, y se actualizan sus datos. También para esta sección se realiza una división de cuerpos para que todos los hilos trabajen de forma simultánea sobre una sección de estos, evitando condiciones de carrera o colisiones sobre la memoria.

Métricas:

Para cada tamaño de unidades de procesamiento y para cada tamaño de problema se toman los tiempos de ejecución promedio de tres iteraciones para realizar un cálculo más realista.

Tiempos	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	11,32	45,44	181,35	728,67
4	4,90	19,12	75,82	304,22
8	2,59	9,86	38,42	153,65
16	1,55	5,18	19,57	77,71

Speedup	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1,00	1,00	1,00	1,00
4	2,31	2,38	2,39	2,40
8	4,36	4,61	4,72	4,74
16	7,32	8,77	9,27	9,38

Eficiencia	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1,00	1,00	1,00	1,00
4	0,58	0,59	0,60	0,60
8	0,55	0,58	0,59	0,59
16	0,46	0,55	0,58	0,59

El análisis de eficiencia de la versión híbrida basada en MPI + Pthreads muestra un comportamiento que se ajusta principalmente al modelo de escalabilidad débil. Al aumentar de forma proporcional el tamaño del problema y la cantidad de núcleos de procesamiento, la eficiencia se mantiene constante como puede verse en las diagonales de la tabla que por ejemplo para 512 cuerpos con 4 núcleos tiene la misma eficiencia que para 1024 cuerpos con 8 núcleos y que para 2048 cuerpos con 16 núcleos.

Por otra parte, también puede analizarse este comportamiento desde la perspectiva de la Ley de Amdahl, ya que para cada tamaño de problema se evaluó la eficiencia frente a un incremento en el número de unidades de procesamiento (por ejemplo, $N = 512$ con 4, 8 y 16 núcleos). Si bien la cantidad de puntos analizados es limitada, los resultados muestran una disminución de eficiencia al aumentar los núcleos con N fijo, especialmente en tamaños pequeños, lo que refleja el impacto del overhead de comunicación y de la fracción secuencial del código. Este efecto es coherente con las predicciones de Amdahl, que establece un límite superior al speedup posible basado en el componente no paralelizable del algoritmo. Aun así, tal como dice la ley de Gustafson al aumentar el tamaño del problema las pérdidas de eficiencia se disminuyen, ya que la parte paralelizable del programa tiene un mayor peso, por lo que se logran aprovechar en mayor medida la paralelización realizada.

En conclusión, la eficiencia observada valida que el diseño híbrido no es fuertemente escalable, ya que no se mantiene constante con problema fijo. Sin embargo, al crecer la carga junto al paralelismo, la eficiencia mejora y se estabiliza, cumpliendo claramente con la definición de escalabilidad débil. Este comportamiento confirma que la solución es adecuada para escenarios distribuidos donde se espera escalar hacia problemas de mayor tamaño y volumen computacional.

Verificación de resultados

Al igual que lo realizado con el código de pthreads se realizó un algoritmo secundario que realiza la simulación secuencial y luego la compara con la simulación paralela.

Distancia promedio al secuencial	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	0	0	0	0
4	1,26	1,72	1,43	2,03
8	1,51	1,8	1,47	2,24
16	1,29	1,82	1,34	2,29

En el caso del uso de MPI-Pthreads la distancia promedio obtenida es mayor que la obtenida para el problema en pthreads. Esto es probable que se deba a la resolución de los tipos de datos utilizados para representar las distintas variables del problema. Aun así, sigue siendo un resultado aceptable siendo que en promedio las distancias rondan los 2 puntos de distancia respecto a su contraparte secuencial.

Conclusiones:

A lo largo del trabajo se desarrollaron y evaluaron tres versiones del algoritmo para la simulación del problema de los N cuerpos: una secuencial, una paralela con Pthreads en memoria compartida y una híbrida combinando MPI y Pthreads para entornos distribuidos con múltiples núcleos por máquina. Los resultados obtenidos permiten comparar el comportamiento de cada enfoque en términos de rendimiento, eficiencia y escalabilidad.

La versión secuencial sirvió como base de referencia, permitiendo identificar las partes más costosas computacionalmente (el cálculo de fuerzas y el movimiento de cuerpos), que fueron paralelizadas en las siguientes versiones. Su ejecución directa sobre una única CPU evidenció la necesidad de escalar la solución para abordar simulaciones de gran tamaño con tiempos razonables.

La versión Pthreads, ejecutada sobre arquitecturas de memoria compartida, mostró un rendimiento muy cercano al ideal, con speedup de hasta 7.16 en 8 hilos y eficiencias superiores al 87% para la mayoría de los tamaños de problema. Este comportamiento fue consistente con una estrategia de paralelización eficiente, bajo overhead y buena localidad de memoria. El análisis de las métricas indicó que esta solución cumple con la definición de escalabilidad débil y, en ciertos escenarios, se comporta como cercana a fuertemente escalable, especialmente cuando se incrementan los hilos en problemas de tamaño moderado.

En cuanto a la solución híbrida MPI + Pthreads, su objetivo fue escalar el algoritmo en entornos de múltiples nodos con múltiples núcleos por nodo. Si bien el speedup no fue lineal debido al overhead de coordinación y comunicación entre procesos, se alcanzaron valores mayores a 9 con 16 núcleos en problemas grandes, demostrando que el diseño híbrido es efectivo cuando se aprovecha con una carga significativa.

En resumen, la versión con Pthreads ofrece un rendimiento excelente en entornos de memoria compartida, mientras que la solución híbrida resulta adecuada para escalar en clusters distribuidos cuando se trabaja con volúmenes de datos importantes y se cuenta con la potencia computacional necesaria. Ambas soluciones muestran un diseño robusto y paralelización bien implementada, validando los objetivos del trabajo tanto desde el punto de vista teórico como práctico.